

Tablice w języku JavaScript

Tablice w JavaScript są jednym z najważniejszych i najczęściej używanych typów danych. Oto kompleksowe opracowanie ich funkcji, właściwości i metod.

Spis treści

1. Podstawowe informacje o tablicach	2
2. Cechy tablic w JavaScript.....	2
3. Właściwości tablic	2
4. Podstawowe metody tablic	2
Dodawanie elementów:	2
Usuwanie elementów:	2
Łączenie tablic:	3
Kopiowanie tablicy:.....	3
5. Zaawansowane metody tablic	3
Iteracja:.....	3
Mapowanie:.....	3
Filtracja:	3
Redukcja:	3
Sortowanie:	3
Odwracanie kolejności:.....	3
6. Sprawdzanie zawartości tablicy.....	4
7. Inne funkcje i techniki	4
Rozpakowywanie tablic:.....	4
Destrukturyzacja:	4
8. Iteratory i pętle	4
Pętla for:.....	4
Pętla for...of:	4
9. Tworzenie tablic specjalnych.....	4
Tablica z określoną długością i wartością:	4
Sprawdzanie, czy coś jest tablicą:.....	5
10. Typowe problemy i najlepsze praktyki	5

1. Podstawowe informacje o tablicach

- **Definicja:** Tablica w JavaScript to obiekt służący do przechowywania uporządkowanych danych.
- **Deklaracja:** Tablice można tworzyć za pomocą nawiasów kwadratowych [] lub konstruktora Array.

```
let tablica1 = [1, 2, 3]; // Deklaracja literatem tablicy
let tablica2 = new Array(1, 2, 3); // Deklaracja z użyciem konstruktora
```

2. Cechy tablic w JavaScript

- **Dynamiczna wielkość:** Tablice mogą zmieniać swoją długość w trakcie działania programu.
- **Dowolne typy danych:** Elementy tablic mogą mieć różne typy danych.

```
let tablica = [1, "tekst", true, { klucz: "wartość" }];
```

- **Indeksowanie:** Tablice są indeksowane od 0.
- **Dostęp:** Elementy można odczytywać i modyfikować za pomocą indeksów.

```
console.log(tablica[1]); // Wyjście: "tekst"
tablica[1] = "nowy tekst"; // Modyfikacja elementu
```

3. Właściwości tablic

- `length` – zwraca liczbę elementów w tablicy.

```
let tablica = [1, 2, 3];
console.log(tablica.length); // Wyjście: 3
```

4. Podstawowe metody tablic

Dodawanie elementów:

- `push()` – dodaje element(y) na końcu tablicy.
- `unshift()` – dodaje element(y) na początku tablicy.

```
tablica.push(4); // [1, 2, 3, 4]
tablica.unshift(0); // [0, 1, 2, 3, 4]
```

Usuwanie elementów:

- `pop()` – usuwa ostatni element.
- `shift()` – usuwa pierwszy element.

```
tablica.pop(); // [0, 1, 2, 3]
tablica.shift(); // [1, 2, 3]
```

Łączenie tablic:

- `concat()` – tworzy nową tablicę przez połączenie dwóch lub więcej tablic.

```
let tablica2 = [4, 5];
let nowaTablica = tablica.concat(tablica2); // [1, 2, 3, 4, 5]
```

Kopiowanie tablicy:

- `slice()` – tworzy kopię lub fragment tablicy.

```
let fragment = tablica.slice(1, 3); // [2, 3]
```

5. Zaawansowane metody tablic

Iteracja:

- `forEach()` – wykonuje podaną funkcję dla każdego elementu.

```
tablica.forEach((element) => console.log(element));
```

Mapowanie:

- `map()` – tworzy nową tablicę, przekształcając każdy element.

```
let podwojona = tablica.map((element) => element * 2); // [2, 4, 6]
```

Filtracja:

- `filter()` – tworzy nową tablicę zawierającą elementy spełniające warunek.

```
let parzyste = tablica.filter((element) => element % 2 === 0); // [2]
```

Redukcja:

- `reduce()` – przekształca tablicę do jednej wartości.

```
let suma = tablica.reduce((acc, element) => acc + element, 0); // 6
```

Sortowanie:

- `sort()` – sortuje tablicę (domyślnie jako ciągi znaków).

```
tablica.sort((a, b) => a - b); // [1, 2, 3]
```

Odwracanie kolejności:

- `reverse()` – odwraca kolejność elementów w tablicy.

```
tablica.reverse(); // [3, 2, 1]
```

6. Sprawdzanie zawartości tablicy

- `includes()` – sprawdza, czy tablica zawiera dany element.
- `indexOf()` – zwraca indeks pierwszego wystąpienia elementu.
- `find()` – zwraca pierwszy element spełniający warunek.
- `findIndex()` – zwraca indeks pierwszego elementu spełniającego warunek.

```
console.log(tablica.includes(2)); // true
console.log(tablica.indexOf(2)); // 1
console.log(tablica.find((x) => x > 1)); // 2
console.log(tablica.findIndex((x) => x > 1)); // 1
```

7. Inne funkcje i techniki

Rozpakowywanie tablic:

- Operator spread `...` pozwala na łatwe kopiowanie i manipulację tablicami.

```
let nowaTablica = [...tablica, 4]; // [1, 2, 3, 4]
```

Destrukturyzacja:

- Umożliwia przypisywanie wartości elementów tablicy do zmiennych.

```
let [pierwszy, drugi] = tablica; // pierwszy = 1, drugi = 2
```

8. Iteratory i pętle

Pętla for:

```
for (let i = 0; i < tablica.length; i++) {
  console.log(tablica[i]);
}
```

Pętla for...of:

```
for (let element of tablica) {
  console.log(element);
}
```

9. Tworzenie tablic specjalnych

Tablica z określoną długością i wartością:

- `Array.from()` – tworzy tablicę z dowolnego obiektu iterowalnego.
- `Array(n).fill()` – tworzy tablicę wypełnioną określoną wartością.

```
let pustaTablica = Array(5).fill(0); // [0, 0, 0, 0, 0]
let tablicaZLiter = Array.from("hello"); // ["h", "e", "l", "l", "o"]
```

Sprawdzanie, czy coś jest tablicą:

- `Array.isArray()`

```
console.log(Array.isArray(tablica)); // true
```

10. Typowe problemy i najlepsze praktyki

- **Mutacja tablicy:** Wiele metod, takich jak `push`, `pop`, `splice`, zmienia oryginalną tablicę. Jeśli potrzebujesz niemutowalnych operacji, używaj metod takich jak `slice` lub `concat`.
- **Porównywanie tablic:** Tablice w JavaScript są obiektami, więc porównywanie dwóch tablic (`==` lub `===`) porównuje ich referencje, a nie zawartość.

```
let a = [1, 2];
let b = [1, 2];
console.log(a === b); // false
```